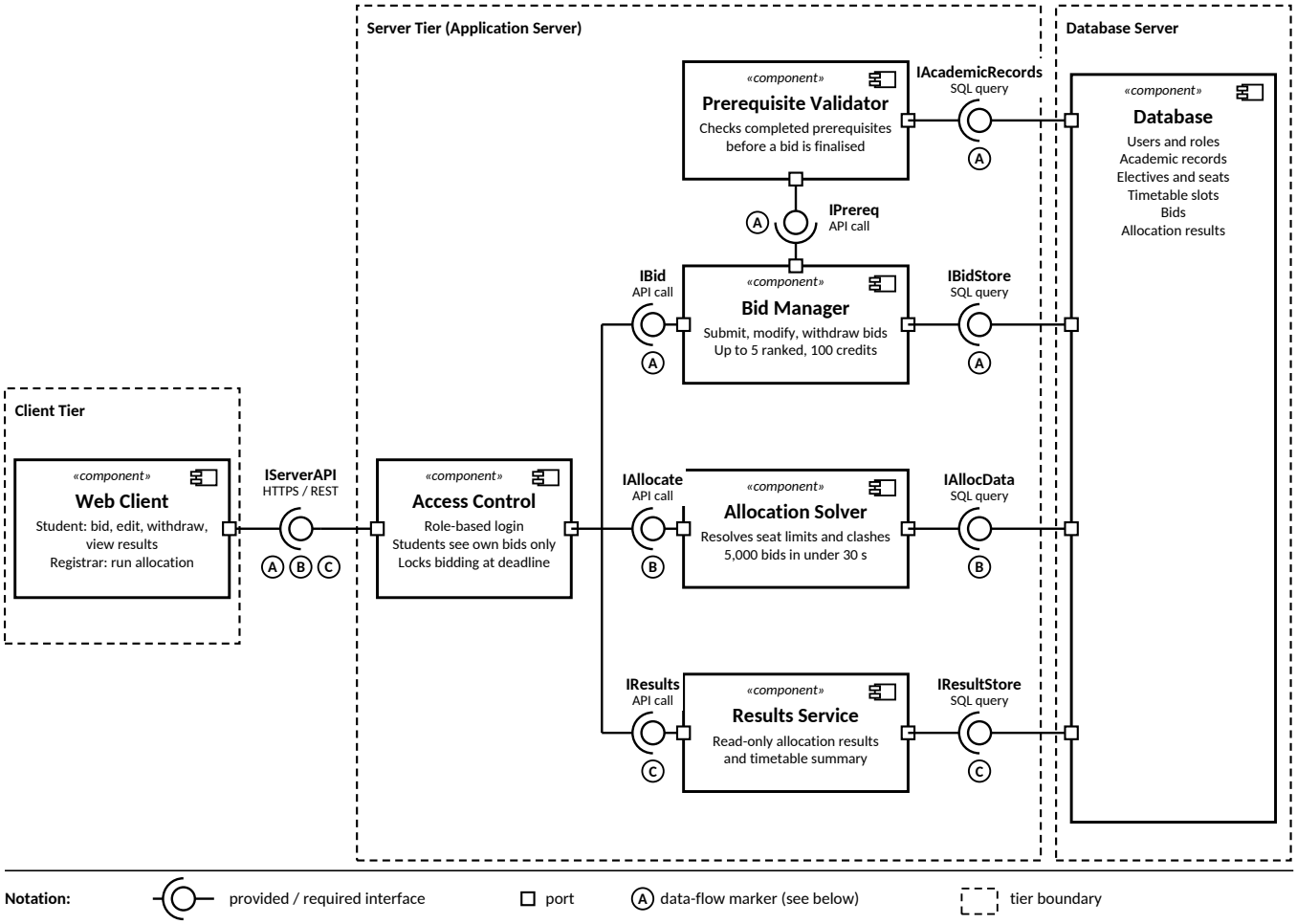
Lab 3: Component Modelling & Architectural Pattern

Team 05: Academic Elective Bidding and Allocation System

PES1UG24AM156

Academic Elective Bidding & Allocation System (AEBA): Client-Server Architecture

UML Component Diagram | Team 05



Data flow

A	Bidding: Web Client → Access Control → Bid Manager → Prerequisite Validator → Database (save bid)
B	Allocation: Registrar (Web Client) → Access Control → Allocation Solver → Database (read bids, write results)
C	Results: Web Client → Access Control → Results Service → Database (read final allocation)

Interfaces

IServerAPI: HTTPS / REST requests from the client	**IAcademicRecords**: SQL: completed courses of a student	**IAllocData**: SQL: read bids, seats, timetable; write results
IBid: submitBid(), modifyBid(), withdrawBid()	**IBidStore**: SQL: save / update / withdraw a bid	**IResults**: getAllocation(student) (read-only)
IPrereq: checkPrerequisites(student, course)	**IAllocate**: runAllocation() (registrar only)	**IResultStore**: SQL: read final allocation

Architecture Selection and Justification

System: Academic Elective Bidding & Allocation System (AEBA)
Selected Architectural Pattern: Client-Server Architecture

1. Architectural Choice

We went with the **Client-Server architecture** for AEBA. Students and the registrar use the Web Client in their browser. All the business logic (Access Control, Bid Manager, Prerequisite Validator, Allocation Solver and Results Service) runs on one application server, and the data sits in a database on a separate database server. The client

never talks to the database. Every request first goes to Access Control through the IServerAPI interface, and Access Control passes it on to the right component.

We also considered the Layered and Microservices patterns. Layered would have worked, but it doesn't say where the heavy allocation work should run. Microservices would split bids and results across several services, and for a system this size that makes it harder to keep the data consistent.

2. Reasons for Selection

Reason 1: Data stays in one place. FR-001 needs exactly 100 credits spread over at most 5 ranked courses, and FR-003 says seats can never be overbooked. With one server and one database, both checks happen in a single place, so every student is treated the same way and there are no copies of the data to keep in sync.

Reason 2: One server controls the rules and the deadline. FR-005 and NFR-002 say a bid cannot be changed after the deadline. Because the server uses its own clock to lock bidding, a student can't get around it by editing the page in the browser. The same server also runs the prerequisite check (FR-002) every time a bid is submitted or changed. It is also easy to deploy, since there is only one server to host and update.

3. Security Advantage

Everything goes through Access Control, which checks the user's role before doing anything else (NFR-002). A student can only see and change their own bids, only the registrar can start the allocation, and the database is never exposed to the browser.

4. Performance Benefit

The Allocation Solver runs on the server next to the database, so it reads all the bids, seat limits and timetable data locally and doesn't have to move them over the network. This helps with NFR-001 (5,000 bids in under 30 seconds). The server can also be given extra CPU and memory for the allocation run at the end of the bidding window.

Conclusion

Client-Server suits AEBA because it keeps the bidding data and rules in one controlled place and keeps the heavy allocation work on the server. The main downside is that the server is a single point of failure, which we would reduce with regular database backups. Allocation only runs once after the deadline, so the load is easy to predict.